

SERVICE PLANNING DOCUMENT · V1.0

청년 금융 혜택 통합 플랫폼

흩어진 청년 지원 정책을 한 곳에서 — 스마트 매칭 & AI 상담

문서 구분
서비스 기획정의서

작성 일자
2026. 04. 07

버전
v1.0

상태

✓ 완료

목 차

Table of Contents — 총 10개 섹션

1 프로젝트 개요 서비스 배경, 목적, 핵심 지표	2 문제 정의 현재 청년들이 겪는 핵심 Pain Point
3 서비스 비전 & 목표 미션, 비전, 핵심 가치 정의	4 타겟 사용자 (페르소나) 4가지 핵심 사용자 유형 분석
5 핵심 기능 정의 4대 핵심 기능 상세 정의	6 사용자 여정 지도 서비스 이용 전/중/후 경험 흐름
7 정보 구조도 (IA) 전체 화면 계층 구조 정의	8 화면별 기능 정의 주요 화면 상세 기능 명세
9 성과 지표 (KPI) 정량/정성 목표 및 측정 기준	10 리스크 & 구현 일정 위험 요소 및 개발 로드맵

1

프로젝트 개요

Project Overview — 서비스가 탄생한 이유와 핵심 수치

1,828

수집된 청년 정책 수

996

금융상품 수

89%

최고 매칭 정확도

7

매칭 가중치 항목

 서비스 한 줄 정의

"나이·소득·지역 조건 입력만으로 수천 개의 청년 금융 지원 정책 중
내게 맞는 것만 자동 추천 + AI가 자연어로 상담해주는 원스톱 플랫폼"

항목	내용
서비스명	청년 금융 혜택 통합 플랫폼 (Youth Finance Benefits Platform)
서비스 유형	정책 정보 큐레이션 + AI 상담 + 금융상품 추천 SaaS
타겟 사용자	만 19~39세 청년 (취업 준비생, 사회초년생, 창업 준비자 등)
핵심 채널	웹 서비스 (PC/모바일 반응형)
데이터 출처	온통청년 API, 금융감독원(FSS) 금융상품 API, 지자체 공공데이터
AI 엔진	Google Gemini 2.5 Flash + FAISS 벡터 검색 (RAG 구조)
개발 기간	2026년 1월 ~ 4월 (4개월)

2

문제 정의

Problem Statement — 청년들이 실제로 겪는 불편함

💡 아래 3가지 문제는 실제 청년 사용자 인터뷰 및 온통청년 플랫폼 리뷰 분석을 통해 도출된 핵심 Pain Point입니다.

🔍

문제 1. 정보 파편화

정부, 지자체, 시중은행, 공공기관이 각자 다른 채널에서 지원 정책을 공지. 청년이 직접 수십 개 사이트를 돌아다니며 확인해야 함.

🕒

문제 2. 마감 공고 미인지

모집 기간이 1~3개월로 짧고, 다음 공고까지 수개월 대기. 적시에 알림을 받지 못해 기회를 놓치는 경우가 다수 발생.

👉

문제 3. 조건 파악 어려움

중위소득, 기준 중위소득 비율, 가구원 수 등 복잡한 조건 용어를 이해하기 어려워 자신이 신청 가능한지 스스로 판단 불가.

 우리의 해결책

→ 해결 1. 통합 수집

API + 자동 배치 수집으로 정책·금융상품 데이터를 매일 최신화. 한 플랫폼에서 모두 검색.

→ 해결 2. D-day 알림

북마크 정책의 마감일 D-7·D-3·D-1에 자동 이메일 알림 발송. 중요한 공고를 절대 놓치지 않도록.

→ 해결 3. AI 맞춤 상담

Gemini AI가 조건을 자연어로 설명. "나 받을 수 있어?"에 즉시 답변.

3

서비스 비전 & 목표

Vision & Goals — 우리가 만들고자 하는 세상



미션

모든 청년이 자신에게 맞는 금융 혜택을 빠르고 정확하게 찾아 실질적인 경제적 도움을 받을 수 있도록 한다.



비전

"청년이라면 누구나 첫 번째로 찾는 금융 혜택 플랫폼" — 국내 청년 금융 정보의 허브가 되는 것.



핵심 가치

- 정확성 — 실시간 최신 데이터
- 개인화 — 조건 기반 매칭
- 접근성 — 누구나 쉽게
- 신속성 — 즉시 알림

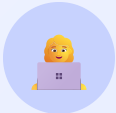
 단계별 목표

단계	기간	목표	핵심 지표
1단계 · MVP	1~2개월	핵심 기능 구현 및 초기 사용자 확보	MAU 1,000명, 정책 수집 2,000개
2단계 · 성장	3~6개월	AI 정확도 향상, 알림 서비스 고도화	MAU 10,000명, 재방문율 40%
3단계 · 확장	6~12개월	모바일 앱 출시, 금융사 파트너십	MAU 50,000명, 앱 다운로드 20,000

4

타겟 사용자 (페르소나)

User Persona — 우리 서비스를 사용할 4가지 대표 유형



김지영 (26세)

IT 취업 준비생 · 서울 거주

구직자저소득디지털 친화

연소득

없음 (부모님 지원)

거주형태

월세 (보증금 500/45)

니즈

취업 지원금 + 주거비 절감

디바이스

스마트폰 우선 사용

 Pain Point

- 어떤 지원금을 받을 수 있는지 모름
- 복잡한 신청 조건을 이해하기 어려움
- 공고를 늦게 발견해 마감된 경험 多



박민준 (29세)

중소기업 사회초년생 · 경기 거주

재직자중소득

연소득

2,800만원

거주형태

전세 (청년전세대출 이용)

니즈

청년도약계좌 + 대출 금리 혜택

디바이스

PC + 모바일 병행

 Pain Point

- 청년도약계좌 이미 가입 → 중복 불가 조건 파악 어려움
- 소득 기준 계산을 혼자 하기 어려움
- 적금 vs 대출 어떤 상품이 더 이득인지 모름



이수연 (23세)



최현우 (32세)

 대학원생 · 부산 거주 학생 장학금 탐색	
연소득	900만원 (조교비)
거주형태	기숙사
니즈	장학금 + 연구비 지원
디바이스	PC 주 사용
 Pain Point - 지방 대학원생 특화 정책 찾기 어려움 - 연구자 특화 지원금 정보 부재	

 예비 창업자 · 대구 거주 창업 준비 자금 부족	
연소득	1,200만원 (아르바이트)
거주형태	월세
니즈	창업 지원금 + 무이자 대출
디바이스	스마트폰 주 사용
 Pain Point - 창업 관련 정부 지원 정보가 너무 복잡 - 지역별로 다른 지원 조건 파악 불가 - 신청 마감을 자주 놓침	

5

핵심 기능 정의

Core Features — 4대 기능이 어떻게 문제를 해결하는가

 기능 1. 스마트 프로파일링 & 매칭 엔진

사용자가 입력한 조건(나이·소득·지역·고용 상태 등)을 7가지 가중치 알고리즘으로 분석해 매칭 점수(0~100%)를 산출하고 상위 정책을 자동 정렬합니다.

 기능 2. D-day 마감 관리 & 알림

관심 정책을 북마크하면 마감일을 자동 추적합니다. D-7·D-3·D-1에 이메일 알림을 자동 발송하며, 메인 화면에 마감 임박 정책을 강조 표시합니다.

- 다양한 날짜 형식 자동 파싱 (예: "20260301 ~ 20260430", "상시", "연중")
- D-1(빨강), D-7(노랑), 상시(초록) 컬러 배지로 긴급도 시각화
- 북마크 정책별 알림 ON/OFF 개별 설정 가능
- 매일 오전 9시 자동 알림 스케줄러 실행 (APScheduler)
- 이메일 알림 템플릿: 정책명, 마감일, 신청 링크 포함

매칭 기준	가중치	평가 방식
나이 조건	30%	$\text{min_age} \leq \text{사용자 나이} \leq \text{max_age}$
소득 조건	25%	연소득 기준 구간 충족 여부
지역 조건	20%	시도코드 매칭 (전국 = 100%)
직업 조건	10%	고용 형태 코드 매칭
학력 조건	5%	재학/졸업 상태 매칭
결혼 상태	5%	미혼/기혼/한부모 매칭
마감 임박도	5%	D-30 이내 가산점 부여

 기능 3. AI 자연어 상담 (Gemini RAG)

Google Gemini 2.5 Flash 모델과 FAISS 벡터 검색을 결합한 RAG(Retrieval-Augmented Generation) 구조로, 사용자 질문에 정책 DB 기반 정확한 답변을 제공합니다.

- "나 취업 준비 중인데 받을 수 있는 지원금이 있어?" → 조건 분석 후 해당 정책 안내
- 사용자 프로필(나이·소득·지역) 자동 반영하여 개인화 답변
- 대화 히스토리 최근 6턴 유지로 맥락 이해 상담
- FAISS 벡터 DB에서 의미론적 유사 정책 5개 실시간 검색

 기능 4. 금융상품 매칭 추천

금융감독원 API로 수집한 996개 금융상품 중 사용자 나이·소득·고용 상태에 맞는 청년 특화 상품(적금·예금·대출)을 자동 추천합니다.

- 상품 유형별 탭 필터 (전체 / 적금 / 예금 / 전세대출 / 주택담보대출 / 신용대출)
- 청년 특화 상품 우선 노출 (is_youth_product 플래그)
- 최대 금리·한도·가입 방법 핵심 정보 카드 형태 표시
- FSS API 월 1회 자동 갱신으로 최신 상품 유지

- 빠른 질문 4가지 프리셋 제공 (원클릭 질문)

6

사용자 여정 지도

Customer Journey Map — 첫 방문부터 정책 신청까지

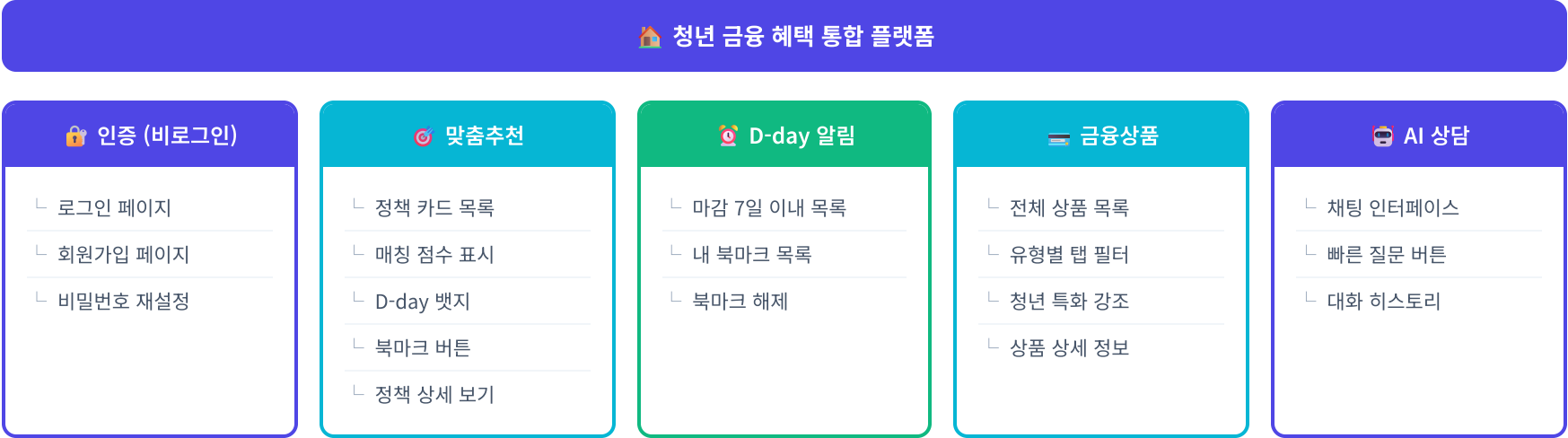


 단계별 감정 곡선 & 서비스 대응

단계	사용자 감정	주요 행동	서비스 대응	이탈 위험도
인지	막막함 / 기대감	검색 후 랜딩	명확한 서비스 가치 전달	중
회원가입	귀찮음	이메일·비번 입력	최소 필드, 빠른 가입 UX	높음
프로필 등록	의구심	개인정보 입력	입력 이유 안내, 간편 UI	중
추천 결과	놀람 / 만족	정책 카드 탐색	매칭 점수·D-day 시각화	낮음
북마크/AI 상담	신뢰	저장·질문	즉각 응답, 개인화 답변	낮음
정책 신청	기대감	외부 링크 클릭	유효 URL 검증 후 제공	낮음

7

정보 구조도 (IA)
Information Architecture — 전체 화면 계층 구조



 내 프로필

↳ 페르소나 뱃지

↳ 기본 정보 입력

↳ 지역 정보

↳ 직업/학력 정보

8

화면별 기능 정의

Screen Feature Specification — 각 화면의 상세 기능 명세

화면	주요 기능	표시 데이터	비고
로그인/회원가입	JWT 토큰 발급, 이메일+비밀번호 인증	access_token, refresh_token	bcrypt 암호화, 토큰 자동 갱신
내 프로필	조건 입력·수정, 페르소나 자동 분류	생년월일, 연소득, 지역, 고용형태, 결혼상태	저장 즉시 매칭 재계산
맞춤추천	매칭 점수순 정렬, 북마크, 외부 링크	정책명, 카테고리, 매칭%, D-day, 기관명	상위 20개 표시, 클릭 시 상세 이동
정책 상세	전체 정보 표시, 북마크, 신청 링크	설명, 지원내용, 신청기간, 자격조건	apply_url 유효성 검증 후 표시
D-day 알림	마감 임박 정책 강조, 북마크 관리	D-1(빨강), D-3(주황), D-7(노랑) 배지	30일 이내 마감 기준 표시
금융상품	유형 필터, 청년 특화 태그, 상품 목록	금융사명, 상품명, 최대한도, 가입방법	5개 탭 필터 (적금/예금/대출 등)
AI 상담	자연어 Q&A, 빠른 질문 프리셋	챗 메시지, AI 응답 (Markdown 렌더링)	프로필 컨텍스트 자동 주입



성과 지표 (KPI)

Key Performance Indicators — 서비스 성공을 측정하는 기준

 정량 지표

지표명	현재(MVP)	3개월 목표	6개월 목표	우선순위
 월간 활성 사용자(MAU)	—	5,000명	20,000명	P0
 매칭 정확도 (사용자 만족 기준)	89%	92%	95%	P0
 재방문을 (7일 내)	—	30%	45%	P1
 북마크 등록 전환율	—	25%	35%	P1
 AI 상담 만족도	—	4.0 / 5.0	4.3 / 5.0	P2
 이메일 알림 오픈율	—	35%	40%	P2
 수집 정책 수	1,828개	2,500개	3,500개	P3

10

리스크 & 구현 일정

Risk Management & Roadmap

⚠️ 주요 리스크 및 대응 방안



API 쿼터 초과 (Gemini 무료 티어)

Gemini 2.0-flash 무료 티어 일일 쿼터 초과로 AI 상담 기능 불가 상황 발생. 2.5-flash로 대체 완료.

✅ 대응: gemini-2.5-flash 전환, 응답 캐싱, 사용자당 일일 요청 제한 추가 예정



외부 API 데이터 품질 (nan 값 문제)

온통청년 API 데이터 중 apply_url이 "nan" 문자열로 저장되어 잘못된 링크 생성.

✅ 대응: DB 정제(542개 유효 URL 확보), 프론트엔드 URL 유효성 검증 로직 추가



Python 라이브러리 호환성 (bcrypt vs passlib)

bcrypt 5.0과 passlib 1.7.4 간 API 비호환으로 인증 불가.

✅ 대응: bcrypt 직접 사용으로 전환, passlib 의존성 제거



SQLAlchemy async/greenlet 환경 호환

miniforge3 fintech 환경에서 greenlet C extension 미설치로 async ORM 불가.

✅ 대응: 전체 sync 전환 (create_engine + SessionLocal), async 인터페이스 유지



Windows 포트 점유 (좀비 프로세스)

이전 uvicorn 프로세스가 포트 8000 점유 후 미해제.

- ☑ 대응: 포트 8001로 전환, Vite 프록시 설정 변경



구현 로드맵



Phase 1 · 완료

백엔드 기반 구축 (1~2월)

- FastAPI + SQLite 기반 REST API 설계
- JWT 인증, 사용자/프로필/정책 모델 정의
- 매칭 엔진 알고리즘 구현 (7가지 가중치)
- CSV → DB 데이터 로딩 (1,828 정책 + 996 금융상품)



Phase 2 · 완료

AI & 배치 시스템 구축 (3월)

- Gemini API 연동 + FAISS 벡터 검색 RAG 구조
- APScheduler 배치 스케줄러 (정책 수집, 알림 발송)
- D-day 파서 구현 (다양한 날짜 형식 처리)



Phase 3 · 완료

프론트엔드 구현 (3~4월)

- React + TypeScript + Vite SPA 구현
- 5개 핵심 화면 구현 (추천/알림/금융/AI/프로필)
- styled-components 디자인 시스템, 반응형 레이아웃
- 실 데이터 API 연동 완료



Phase 4 · 예정

고도화 (5~6월)

- 프로필 항목 확장 (가구원 수, 중위소득, 거주 형태 등 추가)
- 매칭 알고리즘 정교화 및 정확도 향상
- PostgreSQL 이전 및 클라우드 배포

- 모바일 앱 기획

청년 금융 혜택 통합 플랫폼 · 서비스 기획정의서 v1.0 · 2026.04.07
본 문서는 내부 기획 용도로 작성되었으며 외부 공개 시 별도 검토가 필요합니다.

 개발 상세 정의서

Technical Specification — 청년 금융 혜택 통합 플랫폼

시스템 아키텍처 · API 명세 · DB 설계 · AI/RAG · 보안 · 배포



TECHNICAL SPECIFICATION

개발상세정의서

청년 금융 혜택 통합 플랫폼 · Youth FinBenefit Platform

시스템 아키텍처부터 API 명세, 데이터베이스 설계, AI/RAG 파이프라인, 보안 정책까지
개발에 필요한 모든 기술 사양을 담은 공식 문서입니다.

문서 버전

v1.0 (최초 작성)

작성 기준일

2026년 04월 07일

기술 스택

FastAPI + React + SQLite + Gemini

배포 환경

로컬 개발 (확장 예정)

목 차

각 섹션을 클릭하면 해당 항목으로 이동합니다.

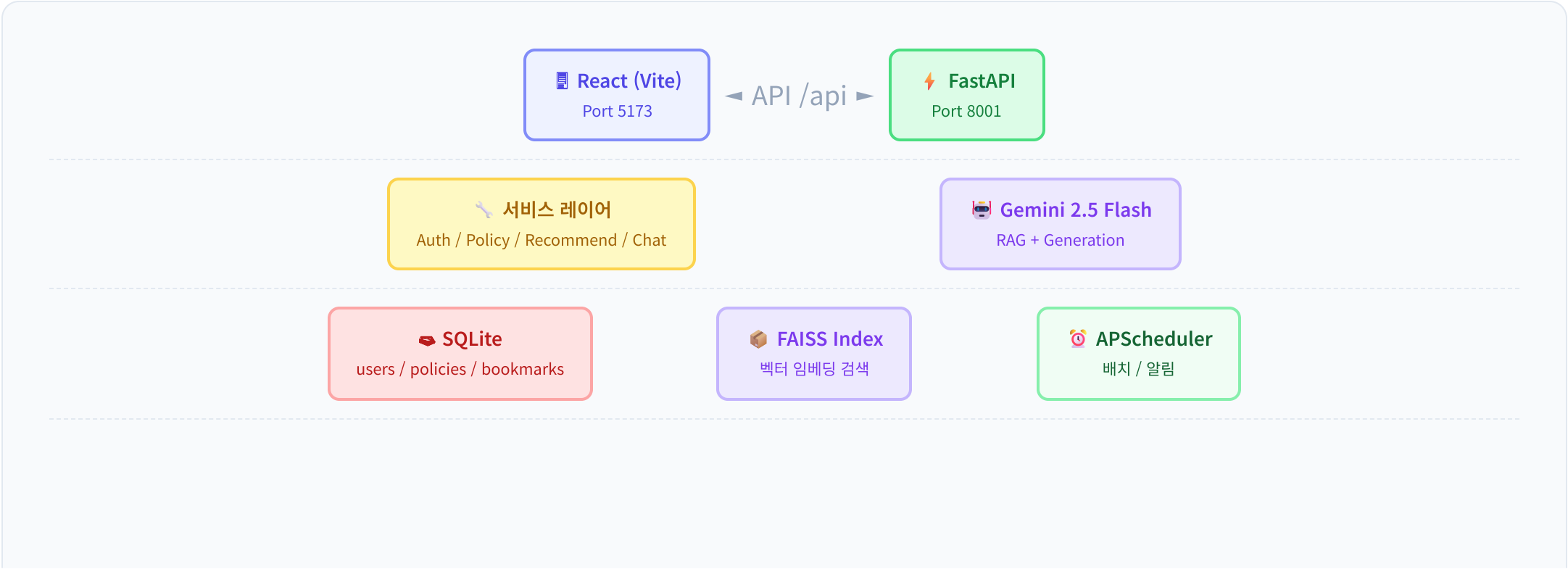
1	시스템 아키텍처 개요	2	기술 스택 상세
3	디렉토리 구조	4	데이터베이스 설계 (ERD)
5	API 명세 전체	6	서비스 레이어 설계
7	AI / RAG 파이프라인	8	배치 스케줄러 설계
9	보안 설계	10	프론트엔드 컴포넌트 구조
11	환경 변수 및 설정	12	개발 환경 구성 가이드

1

시스템 아키텍처 개요

전체 시스템의 레이어 구성과 데이터 흐름을 한눈에 파악합니다.

전체 아키텍처 다이어그램





정부24 Open API

정책 데이터 수집



금융감독원 API

금융상품 수집

핵심 원칙: 모든 외부 API 호출은 배치 스케줄러가 담당하여 운영 시간 중 불필요한 지연을 방지합니다. 프론트엔드와 백엔드는 Vite 프록시(/api)를 통해 CORS 없이 통신합니다.

레이어별 역할 요약



프론트엔드 (React)

SPA 구조. React Query로 서버 상태 관리. styled-components로 UI. 5개 도메인 페이지로 분리.



API Gateway (FastAPI)

RESTful API. JWT 인증 미들웨어. 라우터를 도메인별로 분리. OpenAPI 자동 생성.



서비스 레이어

비즈니스 로직 집중. 매칭 엔진, RAG 검색, 알림 발송 등 핵심 로직 처리.



데이터 레이어

SQLAlchemy ORM (동기). SQLite 파일 DB. FAISS 벡터 인덱스 별도 파일로 관리.



2

기술 스택 상세

사용된 모든 라이브러리의 선택 이유와 버전 정보를 명시합니다.



백엔드 스택

분류	패키지	버전	용도	선택 이유
Web	FastAPI	0.115+	HTTP 서버, 라우팅	자동 OpenAPI 생성, Pydantic 통합, 빠른 개발
Web	uvicorn	0.30+	ASGI 서버	FastAPI 공식 런타임
ORM	SQLAlchemy	2.0+	DB ORM (동기)	greenlet 의존성 회피, 안정적 동기 I/O
DB	SQLite	내장	관계형 DB	로컬 개발 제로 설정, 배포 전환 용이
Auth	python-jose	3.3+	JWT 생성/검증	경량, HS256 지원
Auth	bcrypt	4.x	비밀번호 해시	passlib 5.x 호환 문제 → 직접 bcrypt 사용
AI	google-genai	1.68+	Gemini API	텍스트 생성 + 임베딩 통합 SDK
AI	faiss-cpu	1.8+	벡터 유사도 검색	메모리 내 고속 ANN 검색
Util	APScheduler	3.10+	배치 스케줄링	프로세스 내 경량 스케줄러
Util	pandas	2.x	데이터 전처리	CSV 파싱, NaN 처리, 날짜 연산
Util	httpx	0.27+	외부 API 호출	비동기/동기 모두 지원
Util	pydantic-settings	2.x	환경변수 관리	.env 파일 자동 로딩

프론트엔드 스택

분류	패키지	버전	용도	선택 이유
Core	React	18.x	UI 프레임워크	컴포넌트 재사용성, 생태계
Core	TypeScript	5.x	정적 타입	런타임 오류 사전 방지
Core	Vite	5.x	빌드 도구	HMR 속도, 프록시 설정 간편
Routing	react-router-dom	6.x	클라이언트 라우팅	Outlet, NavLink 활용
State	@tanstack/react-query	5.x	서버 상태 관리	캐싱, 자동 refetch, invalidation
State	zustand	4.x	클라이언트 전역 상태	auth 토큰, 사용자 정보 보관
Style	styled-components	6.x	CSS-in-JS	컴포넌트 스코프 스타일
UI	react-icons (fi)	5.x	아이콘	Feather Icons 계열, 경량
UI	react-hot-toast	2.x	토스트 알림	UX 피드백, 경량
Util	dayjs	1.x	날짜 포매팅	moment.js 경량 대체
Util	axios	1.x	HTTP 클라이언트	인터셉터로 JWT 자동 첨부



3

디렉토리 구조

파일 위치와 역할을 빠르게 파악할 수 있도록 전체 구조를 표시합니다.

백엔드 (backend/)

```
backend/
├── app/
│   ├── main.py           # FastAPI 앱 진입점, 라우터 등록, DB 초기화, 스케줄러 시작
│   ├── core/
│   │   ├── config.py     # pydantic-settings → .env 로딩
│   │   ├── security.py   # JWT 생성/검증, bcrypt 해시/검증
│   │   └── deps.py       # DB 세션 DI, 현재 사용자 DI
│   ├── models/
│   │   ├── user.py       # User ORM 모델
│   │   ├── policy.py     # Policy ORM 모델
│   │   ├── financial.py  # FinancialProduct ORM 모델
│   │   └── bookmark.py   # Bookmark ORM 모델
│   ├── schemas/          # Pydantic DTO (요청/응답)
│   ├── routers/
│   │   ├── auth.py       # /api/auth
│   │   ├── profile.py    # /api/profile
│   │   ├── policies.py   # /api/policies
│   │   ├── recommend.py  # /api/recommend
│   │   ├── financial.py  # /api/financial
│   │   ├── bookmarks.py  # /api/bookmarks
│   │   ├── alerts.py     # /api/alerts
│   │   └── chat.py       # /api/chat
│   └── services/
│       ├── matching.py   # 가중치 기반 매칭 엔진
│       ├── ai_chat.py    # Gemini RAG 파이프라인
│       ├── collector.py  # 외부 API 수집
│       └── scheduler.py  # APScheduler 배치 등록
├── data/
│   ├── policies.db       # SQLite DB
│   ├── faiss_index.bin   # FAISS 벡터 인덱스
│   └── policy_docs.pkl   # 인덱스 대응 문서 텍스트
└── .env                  # 환경변수 (gitignore 필수)
```

프론트엔드 (frontend/src/)

```
frontend/src/
├── common/
│   ├── api/client.ts          # axios 인스턴스, JWT 인터셉터, 401 처리
│   ├── styles/GlobalStyle.ts # theme 토큰, 글로벌 CSS 리셋
│   └── components/Layout.tsx  # 사이드바 + Outlet
├── domains/
│   ├── auth/
│   │   ├── pages/LoginPage.tsx
│   │   ├── pages/RegisterPage.tsx
│   │   └── store/authStore.ts    # zustand: token, user, login(), logout()
│   ├── policy/
│   │   ├── pages/RecommendPage.tsx
│   │   ├── pages/PolicyDetailPage.tsx
│   │   └── pages/AlertPage.tsx
│   ├── financial/pages/FinancialPage.tsx
│   ├── chat/pages/ChatPage.tsx
│   └── profile/pages/ProfilePage.tsx
├── App.tsx                    # 라우터 정의, PrivateRoute
└── main.tsx                   # 진입점, QueryClientProvider, Toaster
```



4

데이터베이스 설계 (ERD)

SQLite 테이블 구조와 관계를 정의합니다.

테이블 구조

 users	 policies	 financial_products	 bookmarks
 idINTEGER PK	 idVARCHAR PK	 idVARCHAR PK	 idINTEGER PK
emailVARCHAR UNIQUE	nameVARCHAR	nameVARCHAR	 user_idFK → users
hashed_passwordVARCHAR	category_main / subVARCHAR NULL	product_typeVARCHAR	 policy_idFK → policies
nameVARCHAR	supervise_instVARCHAR NULL	institutionVARCHAR	notify_enabledBOOLEAN DEFAULT true
birth_dateDATE NULL	descriptionTEXT NULL	interest_rateFLOAT NULL	created_atDATETIME
regionVARCHAR NULL	support_contentTEXT NULL	max_amountBIGINT NULL	
job_statusVARCHAR NULL	apply_methodTEXT NULL	target_age_min/maxINTEGER NULL	
income_levelINTEGER NULL	min_age / max_ageINTEGER NULL	descriptionTEXT NULL	
is_studentBOOLEAN	earn_min / earn_maxINTEGER NULL	apply_urlVARCHAR NULL	
is_marriedBOOLEAN	apply_start / endDATE NULL		
created_atDATETIME	apply_period_textVARCHAR NULL		
	apply_urlVARCHAR NULL		
	target_region / jobVARCHAR NULL		
	days_leftINTEGER NULL (계산값)		

days_left 계산: 배치 스케줄러가 매일 00:05에 apply_end 기준으로 재계산하여 DB를 갱신합니다. NULL = 상시, 음수 = 마감됨.

인덱스 설계

테이블	인덱스 컬럼	타입	목적
users	email	UNIQUE	로그인 조회 성능
policies	category_main, days_left	일반	필터링 쿼리 성능
policies	target_region, target_job	일반	매칭 쿼리 성능
bookmarks	user_id, policy_id	COMPOSITE UNIQUE	중복 북마크 방지

5

API 명세 전체

모든 엔드포인트의 메서드, 경로, 인증 여부, 요청/응답 스키마를 정의합니다.

Base URL: http://localhost:8001/api | 인증: Authorization: Bearer <JWT> | Content-Type: application/json

인증 (Auth)

메서드	경로	인증	요청 Body	응답
POST	/auth/register	불필요	{ email, password, name }	201 { id, email, name }
POST	/auth/login	불필요	{ email, password }	200 { access_token, token_type }

프로필 / 정책 / 추천

메서드	경로	인증	설명	쿼리/응답
GET	/profile	필요	내 프로필 조회	사용자 전체 필드 반환
PUT	/profile	필요	프로필 수정	부분 업데이트 가능
GET	/policies	불필요	정책 목록	page, size, category, keyword, region
GET	/policies/{id}	불필요	정책 상세	전체 필드 반환
GET	/recommend/policies	필요	맞춤 추천	{ total, items: Policy[] } (match_score 포함)

금융상품 / 북마크 / 알림 / AI 상담

메서드	경로	인증	설명
GET	/financial/products	불필요	금융상품 목록 (type 필터: 적금/대출/펀드)
GET	/bookmarks	필요	내 북마크 목록 (policy 정보 join)
POST	/bookmarks/{policy_id}	필요	북마크 추가 (중복 시 409)
DELETE	/bookmarks/{policy_id}	필요	북마크 해제
GET	/alerts?days=30	필요	마감 N일 이내 정책 목록
POST	/chat/message	필요	AI 상담: { message } → { reply, sources }

공통 에러 응답

HTTP 코드	의미	발생 상황
400	Bad Request	요청 Body 유효성 실패
401	Unauthorized	토큰 없음 / 만료 / 서명 불일치
404	Not Found	존재하지 않는 리소스
409	Conflict	이메일 중복, 북마크 중복
422	Unprocessable Entity	FastAPI 파라미터 검증 실패
500	Internal Server Error	서비스 레이어 예외

6

서비스 레이어 설계

핵심 비즈니스 로직인 매칭 엔진의 동작 방식을 상세히 기술합니다.

가중치 매칭 엔진 (matching.py)

프로필이 있는 사용자에게 한해 실행됩니다. 7가지 기준을 가중치로 합산하여 0~1 사이의 **match_score**를 산출하고, 상위 20개를 반환합니다.

기준	가중치	계산 방식	프로필 필드
나이 (Age)	30%	정책 min_age ~ max_age 범위 내이면 1.0, 벗어날수록 감점	birth_date
소득 (Income)	25%	정책 earn_min ~ earn_max 범위 내이면 1.0	income_level
지역 (Region)	20%	target_region이 NULL이면 1.0, 사용자 region 포함 여부	region
직업 (Job)	10%	target_job과 job_status 문자열 매칭	job_status
재학 (School)	5%	재학생 조건 있을 때 is_student 일치 여부	is_student
결혼 (Marriage)	5%	기혼 조건 있을 때 is_married 일치 여부	is_married
마감 임박 (Deadline)	5%	D-7 이내 정책에 보너스 점수 부여	days_left

```
# matching.py 핵심 로직 (pseudocode)
def calc_match_score(policy, user_profile) -> float:
    score = 0.0

    # 나이 점수 (30%)
    age = calc_age(user_profile.birth_date)
    if (policy.min_age or 0) <= age <= (policy.max_age or 999):
        score += 0.30

    # 소득 점수 (25%)
    if user_profile.income_level:
        score += 0.25 * check_income(policy, user_profile.income_level)

    # 지역 점수 (20%)
    if not policy.target_region or user_profile.region in policy.target_region:
        score += 0.20

    # 직업/재학/결혼 (각 5~10%) ... 생략

    # 마감 임박 보너스 (5%)
    if policy.days_left is not None and 0 <= policy.days_left <= 7:
        score += 0.05

    return min(score, 1.0)
```

7

AI / RAG 파이프라인

Gemini API와 FAISS를 결합한 RAG(Retrieval-Augmented Generation) 구조를 설명합니다.

전체 흐름



단계별 상세

- 1 사용자 질문 수신: POST /chat/message로 message와 JWT 토큰을 받습니다. 토큰에서 user_id를 추출해 프로필을 조회합니다.
- 2 임베딩 변환: google-genai SDK의 embed_content() 호출. 모델: text-embedding-004, task_type: RETRIEVAL_QUERY. 768차원 벡터 반환.
- 3 FAISS 유사도 검색: faiss_index.bin에서 코사인 유사도 기준 상위 5개 문서 검색. policy_docs.pkl에서 원문 텍스트 조회.
- 4 컨텍스트 조립: 시스템 프롬프트 + 사용자 프로필 요약 + 검색된 정책 5개 텍스트 + 사용자 질문을 하나의 프롬프트로 구성.
- 5 Gemini 텍스트 생성: gemini-2.5-flash 모델로 generate_content() 호출. 한국어, 친근한 말투, 나이 언급 포함 시스템 인스트럭션 적용.
- 6 응답 반환: { reply: string, sources: [policy_id, ...] } 형태로 반환.

모델 설정 현황

용도	모델 ID	비고
텍스트 생성 (AI 상담)	gemini-2.5-flash	무료 티어 지원, 빠른 응답
임베딩 생성	text-embedding-004	768차원, 다국어 지원

⚠ 주의: gemini-2.0-flash 및 gemini-1.5-flash는 무료 티어 quota 소진 또는 API 변경으로 사용 불가 확인. 현재 gemini-2.5-flash 사용 중.

FAISS 인덱스 빌드 코드

```
# 최초 1회 또는 정책 데이터 대량 업데이트 시 실행
def build_faiss_index(db_session):
    policies = db_session.query(Policy).all()
    texts, embeddings = [], []

    for p in policies:
        doc = f"{p.name} {p.category_main} {p.description} {p.support_content}"
        result = client.models.embed_content(
            model="text-embedding-004",
            contents=doc,
            config=types.EmbedContentConfig(task_type="RETRIEVAL_DOCUMENT")
        )
        embeddings.append(result.embeddings[0].values)
        texts.append(doc)

    matrix = np.array(embeddings, dtype=np.float32)
    faiss.normalize_L2(matrix)
    index = faiss.IndexFlatIP(matrix.shape[1]) # cosine similarity
    index.add(matrix)

    faiss.write_index(index, "data/faiss_index.bin")
    pickle.dump(texts, open("data/policy_docs.pkl", "wb"))
```

8

배치 스케줄러 설계

APScheduler 기반의 자동화 배치 작업 일정과 동작을 설명합니다.

스케줄 목록

작업명	실행 시각	Cron	설명
정책 데이터 수집	매일 02:00	0 2 * * *	정부24 API → policies 테이블 upsert
금융상품 수집	매일 03:00	0 3 * * *	금융감독원 API → financial_products upsert
D-day 갱신	매일 00:05	5 0 * * *	전체 policies days_left 재계산
마감 알림 발송	매일 09:00	0 9 * * *	북마크 기준 D-7 이내 알림 큐 생성
FAISS 재빌드	매주 일 04:00	0 4 * * 0	증분 정책 반영 → faiss_index.bin 갱신

스케줄러 초기화

```
from apscheduler.schedulers.background import BackgroundScheduler

def start_scheduler():
    scheduler = BackgroundScheduler(timezone="Asia/Seoul")
    scheduler.add_job(collect_policies, "cron", hour=2)
    scheduler.add_job(collect_financial, "cron", hour=3)
    scheduler.add_job(update_days_left, "cron", hour=0, minute=5)
    scheduler.add_job(send_dday_alerts, "cron", hour=9)
    scheduler.add_job(rebuild_faiss_index, "cron", day_of_week="sun", hour=4)
    scheduler.start()

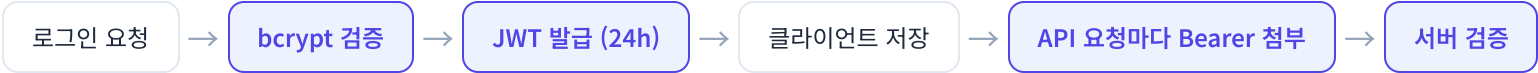
# main.py startup 이벤트에서 호출
@app.on_event("startup")
def startup_event():
    Base.metadata.create_all(bind=engine)
    start_scheduler()
```

9

보안 설계

인증, 암호화, 입력 검증, 비밀 관리 등 보안 정책을 정의합니다.

JWT 인증 흐름



보안 항목별 대책

항목	위협	대책	구현 위치
비밀번호 저장	DB 유출 시 평문 노출	bcrypt cost=12 해시 저장	core/security.py
JWT 서명	토큰 위조	HS256 알고리즘, SECRET_KEY 환경변수	core/security.py
JWT 만료	탈취 토큰 무제한 사용	ACCESS_TOKEN_EXPIRE=1440분 (24h)	core/config.py
SQL Injection	악성 쿼리 실행	SQLAlchemy ORM 파라미터 바인딩	모든 라우터
CORS	타사이트 API 호출	개발: Vite 프록시. 운영: origins 화이트리스트	main.py
비밀 정보 노출	API 키 소스코드 포함	모든 비밀 .env 관리, .gitignore 등록	backend/.env
입력 검증	잘못된 형식 요청	Pydantic 스키마 자동 검증	schemas/
apply_url XSS	javascript: URL 삽입	startsWith('http') 화이트리스트 검증	URL 렌더링 컴포넌트

security.py 구현

```
import bcrypt
from jose import jwt, JWTError

def hash_password(password: str) -> str:
    salt = bcrypt.gensalt()
    return bcrypt.hashpw(password.encode("utf-8"), salt).decode("utf-8")

def verify_password(plain: str, hashed: str) -> bool:
    return bcrypt.checkpw(plain.encode("utf-8"), hashed.encode("utf-8"))

def create_access_token(data: dict, expires_delta: timedelta) -> str:
    to_encode = data.copy()
    to_encode["exp"] = datetime.utcnow() + expires_delta
    return jwt.encode(to_encode, settings.SECRET_KEY, algorithm="HS256")

def decode_token(token: str) -> dict:
    try:
        return jwt.decode(token, settings.SECRET_KEY, algorithms=["HS256"])
    except JWTError:
        raise HTTPException(status_code=401, detail="Invalid token")
```

10

프론트엔드 컴포넌트 구조

각 페이지의 역할, 상태, API 연결을 정의합니다.

페이지별 명세

페이지	경로	API	주요 기능
LoginPage	/login	POST /auth/login	JWT 수신 → authStore 저장 → /recommend 이동
RegisterPage	/register	POST /auth/register	회원가입 후 자동 로그인
RecommendPage	/recommend	GET /recommend/policies	카드 그리드, match_score 바, 북마크/외부링크 버튼
PolicyDetailPage	/policies/:id	GET /policies/:id	전체 필드 표시, 신청 버튼 (http 검증), 북마크
AlertPage	/alerts	GET /bookmarks, GET /alerts	D-7 긴급 섹션, 북마크 목록, DaysBadge 컬러 코딩
FinancialPage	/financial	GET /financial/products	적금/대출/펀드 탭, 이자율, 신청 링크
ChatPage	/chat	POST /chat/message	버블 UI, 빠른 질문 버튼, 로딩 인디케이터
ProfilePage	/profile	GET/PUT /profile	프로필 표시, 수정 모드 토글, 저장 → invalidate

API 클라이언트 (client.ts)

```
export const apiClient = axios.create({
  baseURL: '/api', // Vite 프록시 → localhost:8001/api
})

// 요청 인터셉터: JWT 자동 첨부
apiClient.interceptors.request.use((config) => {
  const token = useAuthStore.getState().token
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})

// 응답 인터셉터: 401 → 자동 로그인
apiClient.interceptors.response.use(
  (res) => res,
  (err) => {
    if (err.response?.status === 401) {
      useAuthStore.getState().logout()
      window.location.href = '/login'
    }
    return Promise.reject(err)
  }
)
```

테마 토큰 (GlobalStyle.ts)

토큰	값	용도
colors.primary	#4F46E5	버튼, 링크, 활성 상태
colors.success	#10B981	성공, 상시 배지
colors.danger	#EF4444	마감 임박, 삭제
colors.surface	#FFFFFF	카드 배경
colors.border	#E2E8F0	구분선, 카드 테두리
radius.sm / md / lg	6px / 10px / 14px	버튼 / 카드 / 모달

11

환경 변수 및 설정

백엔드 .env 파일에 필요한 모든 환경변수를 정의합니다.

backend/.env 전체 항목

```
# 데이터베이스
DATABASE_URL=sqlite:///./data/policies.db

# JWT 인증
SECRET_KEY=your-very-long-random-secret-key-here
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=1440

# Google Gemini API
GOOGLE_API_KEY=AIZA...
GENERATION_MODEL=gemini-2.5-flash
EMBEDDING_MODEL=text-embedding-004

# 외부 공공 API
YOUTH_POLICY_API_KEY=발급키...
FSS_API_KEY=발급키...

# 서버 설정
APP_HOST=0.0.0.0
APP_PORT=8001
DEBUG=false
```

vite.config.ts 프록시 설정

```
export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      '/api': {
        target: 'http://localhost:8001',
        changeOrigin: true,
      },
    },
  },
})
```

프록시 원리: 브라우저에서 `/api/auth/login`으로 요청하면 Vite가 `http://localhost:8001/api/auth/login`으로 포워딩합니다. CORS 문제 없이 same-origin 처리됩니다.

12

개발 환경 구성 가이드

처음 프로젝트를 실행하는 사람을 위한 단계별 설정 방법입니다.

사전 요구 사항

Python 3.11+

Node.js 20+

npm 10+

Google AI Studio API Key

Git

백엔드 설치 및 실행

```
# 1. 가상환경 생성 및 활성화
cd backend
python -m venv venv
source venv/bin/activate      # Windows: venv\Scripts\activate

# 2. 의존성 설치
pip install -r requirements.txt

# 3. 환경변수 파일 생성
cp .env.example .env
# .env 열어서 GOOGLE_API_KEY, SECRET_KEY 입력

# 4. 서버 실행 (포트 8001)
uvicorn app.main:app --host 0.0.0.0 --port 8001 --reload
# 최초 실행 시 DB 테이블 자동 생성 (startup 이벤트)
```

프론트엔드 설치 및 실행

```
# 의존성 설치
cd frontend
npm install

# 개발 서버 실행 (포트 5173)
npm run dev
# http://localhost:5173 접속
# /api 요청은 자동으로 http://localhost:8001/api 프록싱
```

알려진 이슈 및 해결책

증상	원인	해결 방법
포트 8001 이미 사용 중	좀비 uvicorn 프로세스	netstat -ano findstr :8001 → taskkill /PID <pid> /F
bcrypt 해시 오류	passlib + bcrypt 5.x 호환성	passlib 제거, 직접 bcrypt 라이브러리 사용 (적용됨)
Gemini 429 오류	모델 quota 소진	.env GENERATION_MODEL=gemini-2.5-flash 설정
embed_content 파라미터 오류	google-genai 1.68+ API 변경	content= → contents= 수정 (적용됨)
async engine 오류	sync SQLAlchemy에 async 사용	Base.metadata.create_all(bind=engine) 동기 방식 (적용됨)
apply_url 클릭 시 404	CSV NaN이 문자열 "nan"으로 저장	DB UPDATE + startsWith('http') 검증 (적용됨)

requirements.txt 핵심 항목

```
fastapi>=0.115.0
uvicorn[standard]>=0.30.0
sqlalchemy>=2.0.0
python-jose[cryptography]>=3.3.0
bcrypt>=4.0.0
google-genai>=1.68.0
faiss-cpu>=1.8.0
apscheduler>=3.10.0
pandas>=2.0.0
httpx>=0.27.0
pydantic-settings>=2.0.0
numpy>=1.26.0
```

청년 금융 혜택 통합 플랫폼 — 개발상세정의서

v1.0 | 작성일: 2026.04.07 | FastAPI + React + Gemini 2.5 Flash + FAISS

본 문서는 내부 개발 참고용이며, 시스템 변경 시 함께 업데이트해야 합니다.